

FITBUDDY

AI Fitness Plan Generator using Gemini Models

Updated Project Report

Project Type AI-powered web application	Backend FastAPI + Python
AI Layer Google Gemini	Database SQLite + SQLAlchemy

This report uses the supplied project document as the reference and updates the technical structure and screenshots to match the FitBuddy implementation developed during the project work.

Prepared for Project Documentation / Demonstration

1. Table of Contents

No.	Section	Page
1	Project Overview	3
2	Problem Statement, Objectives & Scope	4
3	Major Features & User Scenarios	5
4	Technology Stack & System Architecture	6
5	Updated Project Structure	7
6	Backend Modules & API Flow	8
7	AI Integration & Prompt Workflow	9
8	Frontend – Home Page	10
9	Frontend – Personalized Workout Result	11
10	Admin Dashboard	12
11	Database, Feedback & Data Flow	13
12	Testing, Setup & Deployment	14
13	Conclusion, Limitations & Future Scope	15

Reference basis: The supplied FitBuddy project document describes the project purpose, scenarios, architecture, development milestones, frontend pages, deployment process and conclusion. The implementation details in this report are updated to reflect the project structure and Gemini setup used in the completed application.

2. Project Overview

FitBuddy is a web-based AI fitness planning application designed to generate personalized 7-day workout plans and complementary nutrition or recovery guidance. The reference document describes a system that accepts user information such as name, age, weight, fitness goal and workout intensity, then uses a Gemini model to create a structured plan.

The completed application follows the same core idea while using a modular FastAPI backend, Jinja2 templates for the web interface, SQLite for local persistence and the Google GenAI SDK for Gemini integration. The application also includes feedback-based plan updating and an administrator dashboard for viewing registered users and their stored plans.

The project demonstrates how a generative AI service can be connected to a conventional web application. Instead of hard-coding a separate workout routine for every user, the backend builds a prompt from the submitted profile and asks Gemini to produce a personalized response.

The supplied document identifies four major usage scenarios: personalized workout generation, feedback-based plan refinement, nutrition/recovery guidance, and administrative viewing of users and plans. These scenarios form the functional core of the completed application.

The current implementation was tested locally using Python 3.12, a virtual environment, Uvicorn and the Google GenAI SDK. During development, several Gemini model variants were tested; the working configuration was updated to use Gemini 3.5 Flash Lite for the implemented generation tasks.

Key outcome

- A working browser-based fitness planning interface.
- AI-generated 7-day workout plans based on submitted user information.
- Personalized nutrition and recovery guidance.
- Feedback-driven plan updating.
- SQLite persistence for users and generated plans.
- An administrator page for reviewing registered users.

3. Problem Statement, Objectives & Scope

Problem Statement

Many fitness applications present generic routines that do not directly reflect a user's stated goal, preferred intensity or personal profile. The project addresses this by providing a simple interface through which a user can submit profile information and receive an AI-generated weekly plan. The reference document positions FitBuddy as a way to simplify the creation of structured fitness routines and supporting nutrition/recovery advice.

Objectives

- Collect the minimum profile information needed to personalize a plan.
- Generate a structured 7-day fitness plan using a Gemini model.
- Provide concise nutrition and recovery guidance.
- Allow users to submit feedback and request a revised plan.
- Store user and plan information in a local SQLite database.
- Provide an admin view for monitoring registered users and plans.
- Keep the application modular so AI, routing, database and frontend responsibilities are separated.

Project Scope

Included	Not the primary scope
Personalized 7-day workout generation	Medical diagnosis or treatment
Nutrition/recovery tips	Clinical nutrition prescriptions
Feedback-based plan revision	Real-world biometric monitoring
Local database persistence	Production-scale cloud deployment
Admin user/plan overview	Wearable device integration

4. Major Features & User Scenarios

Scenario 1 – Personalized Plan Generation

The user enters their profile details, fitness goal and preferred intensity. FastAPI validates and processes the form data, the AI generation module builds a prompt, Gemini returns a structured plan, and the result is rendered in the result page.

Scenario 2 – Feedback-Based Updating

The user can provide feedback such as requesting more cardio, more rest or another adjustment. The application retrieves the stored plan, combines it with the feedback and sends the information to the plan-update module. The revised result is stored and displayed.

Scenario 3 – Nutrition & Recovery Guidance

A separate AI generation function produces a concise nutrition or recovery section related to the user's goal, intensity and profile. The completed implementation was adjusted so the response is complete, readable and personalized rather than a short incomplete fragment.

Scenario 4 – Administrator View

The admin dashboard presents registered users and their fitness information in a structured table. The supplied screenshot shows the completed dashboard with user IDs, names, ages, weights, goals, intensity values, creation times and delete actions.

Main Functional Features

- Responsive user-facing home page.
- Personal profile collection through a web form.
- 7-day AI workout generation.
- Nutrition/recovery section.
- Feedback submission and plan update.
- Persistent user and plan storage using SQLite.
- Admin dashboard for registered users.
- API health check and programmatic generation endpoints.
- Local development using Uvicorn with automatic reload.

5. Technology Stack & System Architecture

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript, Jinja2	User interface, forms and dynamic result pages
Backend	Python + FastAPI	Routing, validation and application logic
AI	Google Gemini + Google GenAI SDK	Workout, nutrition and feedback generation
Database	SQLite + SQLAlchemy	Persistent user and plan records
Server	Uvicorn	ASGI development server
Configuration	python-dotenv	.env-based configuration and API key loading
Testing	Python test module / project tests	Basic verification of application behaviour

Logical Architecture

USER	→	Jinja2 / HTML UI	→	FastAPI Routes
Profile + feedback		Form submission / result display		Validation + orchestration
	→	Gemini AI Layer	↔	SQLite / SQLAlchemy
		Workout • Nutrition • Update		Users • Plans • Updates

The reference document describes the same four-layer concept: frontend templates, FastAPI backend, Gemini AI services and SQLite/SQLAlchemy persistence. The implementation is organized into separate Python modules to keep these responsibilities maintainable.

6. Updated Project Structure

```
FitBuddy-AI/
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── database.py
│   ├── schemas.py
│   ├── gemini_client.py
│   ├── gemini_generator.py
│   ├── gemini_flash_generator.py
│   ├── updated_plan.py
│   ├── routes.py
│   └── main.py
├── templates/
│   ├── index.html
│   ├── result.html
│   ├── all_users.html
│   └── error.html
├── static/
│   ├── style.css
│   └── script.js
├── tests/
│   └── test_app.py
├── docs/
│   ├── 01_brainstorming.md
│   ├── 02_requirement_analysis.md
│   ├── 03_project_design.md
│   ├── 04_project_planning.md
│   ├── 05_project_development.md
│   ├── 06_project_testing.md
│   ├── 07_project_documentation.md
│   └── 08_project_demonstration.md
├── .env.example
├── .gitignore
├── requirements.txt
└── README.md
```

Why this structure was updated

- **app/** contains the complete backend and AI integration code.
- **templates/** contains the Jinja2 pages used by the browser interface.
- **static/** contains shared CSS and JavaScript instead of embedding all styling in individual pages.
- **tests/** provides a dedicated place for application tests.
- **docs/** matches the eight phase-wise submission structure requested in the supplied project instructions.
- **.env.example** documents configuration without exposing the real API key.
- **requirements.txt** records the Python dependencies required to run the application.

Runtime note

The SQLite database file is created/used at runtime and should not be treated as source code. The real Gemini API key belongs in the local `.env` file and should never be committed to a public repository.

7. Backend Modules & API Flow

main.py

Creates the FastAPI application, initializes the database through the lifespan hook, mounts the static directory and includes the application router.

routes.py

Connects browser/API requests to the appropriate validation, AI and database functions. It handles home, workout generation, feedback updates and admin views.

schemas.py

Defines structured request models used to validate user input and feedback data.

database.py

Initializes SQLite/SQLAlchemy storage and provides functions for saving and retrieving users and plans.

gemini_client.py

Creates the Gemini client and centralizes text-generation requests so other modules do not duplicate API setup code.

gemini_generator.py

Builds the workout-generation prompt and requests a structured 7-day plan.

gemini_flash_generator.py

Builds the personalized nutrition/recovery prompt and returns the completed guidance section.

updated_plan.py

Uses the stored plan and user feedback to request a revised plan.

config.py

Loads environment variables, model names, database settings and application configuration.

Important routes/endpoints

Route	Purpose
/	Displays the main FitBuddy form.
/generate-workout	Processes user details and generates the workout + nutrition result.
/submit-feedback	Accepts feedback and requests an updated plan.
/view-all-users	Displays the admin dashboard.
/api/health	Returns a simple application health status.
/api/generate-workout	Provides an API-oriented workout generation path.

8. AI Integration & Prompt Workflow

The reference document originally described separate Gemini Pro and Gemini Flash roles. During the actual development process, the available Gemini models were tested directly and the working implementation was updated to Gemini 3.5 Flash Lite. This keeps the architecture simple while using a model that was successfully reachable by the project's API key.

The central Gemini client is responsible for creating the GenAI client from the API key and sending generation requests. Higher-level modules focus on prompts and application-specific instructions rather than duplicating client setup.

Workout generation flow

- Collect name, age, weight, fitness goal and intensity.
- Create a structured prompt describing the requested 7-day plan.
- Send the prompt to the configured Gemini workout model.
- Validate that a non-empty response is returned.
- Store the generated plan and render it in result.html.

Nutrition / recovery flow

- Use the user's profile and goal to build a separate nutrition/recovery prompt.
- Request practical, age-appropriate general wellness guidance.
- Require a complete response instead of an unfinished fragment.
- Display the generated section beneath the workout plan.

Feedback update flow

- Retrieve the original stored plan using the user ID.
- Combine the original plan with the user's feedback.
- Ask Gemini to revise the plan while preserving the user's core goal.
- Save the revised plan separately so the original can still be reviewed.

Configuration used in the working version

```
GEMINI_WORKOUT_MODEL=gemini-3.5-flash-lite
GEMINI_NUTRITION_MODEL=gemini-3.5-flash-lite
ALLOW_DEMO_FALLBACK=true
DATABASE_URL=sqlite:///./fitbuddy.db
MIN_AGE=13
MAX_AGE=100
```

The real API key is intentionally omitted from this report.

9. Frontend – Home Page

The current home page is the main entry point of FitBuddy. It introduces the application as a personal AI fitness coach and presents the core value of the system before the user reaches the input form.

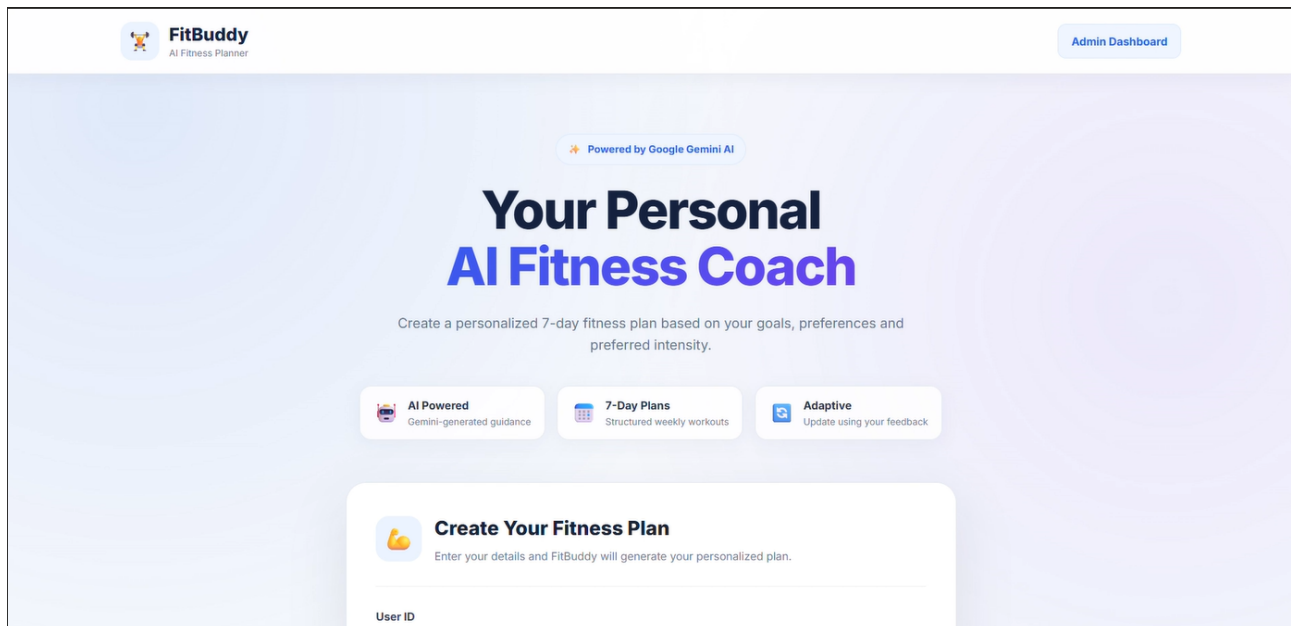


Figure 1. Current FitBuddy home page screenshot supplied for the updated project report.

Visible design elements include the FitBuddy header, Google Gemini AI indicator, AI-powered feature card, 7-day plans feature, adaptive feedback feature and the “Create Your Fitness Plan” section. The screenshot replaces the older/reference document's illustrative home-page image.

The interface is intentionally simple: the user starts by entering the required profile information, submits the form, and then moves to the generated plan page. Shared styling is provided through the project's static CSS file.

10. Frontend – Personalized Workout Result

After successful form submission, FitBuddy displays the information used to personalize the plan followed by the AI-generated 7-day fitness plan. This page is the main output screen of the application.

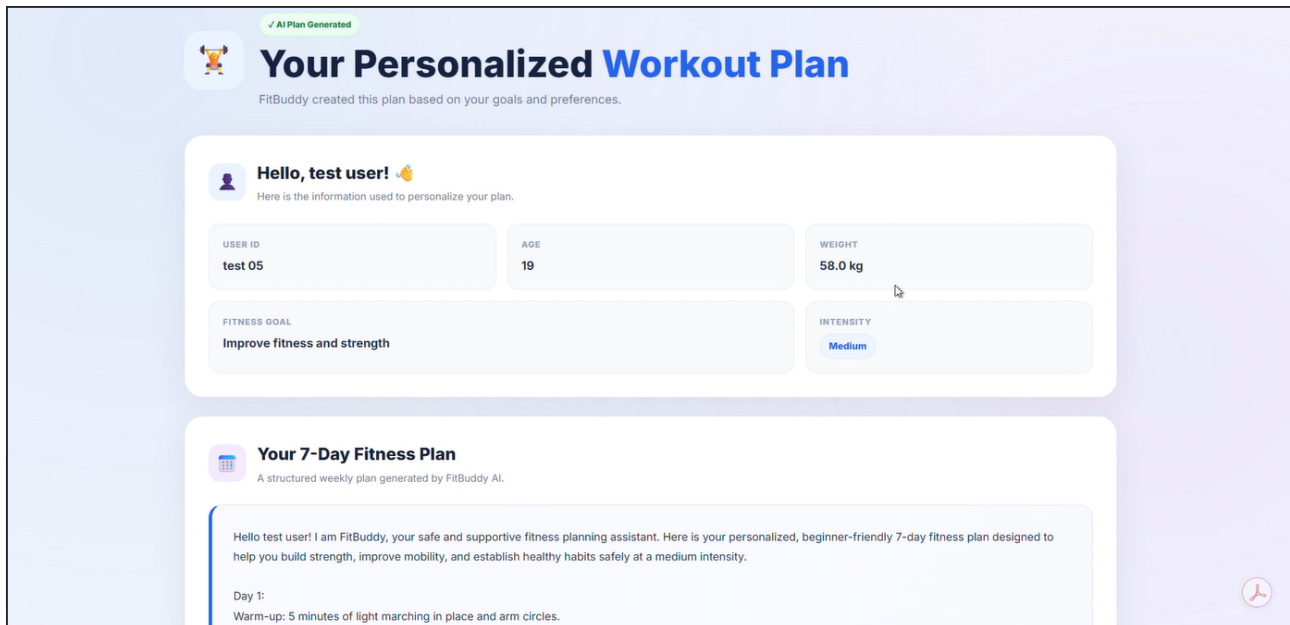


Figure 2. Current personalized workout result screenshot supplied for the updated project report.

The screenshot shows the “AI Plan Generated” status, the personalized greeting, user ID, age, weight, fitness goal and intensity. Below this information, the 7-day fitness plan begins with a structured introduction and Day 1 content. The complete page continues with the remaining plan content and supporting sections.

The result page is dynamically rendered by Jinja2. Values such as the user details, workout plan and nutrition tip are passed from the FastAPI route into the template rather than being hard-coded.

11. Admin Dashboard

FitBuddy includes an administrative view for reviewing registered users and their fitness information. The supplied current screenshot is used here in place of the older/reference document's admin-page image.

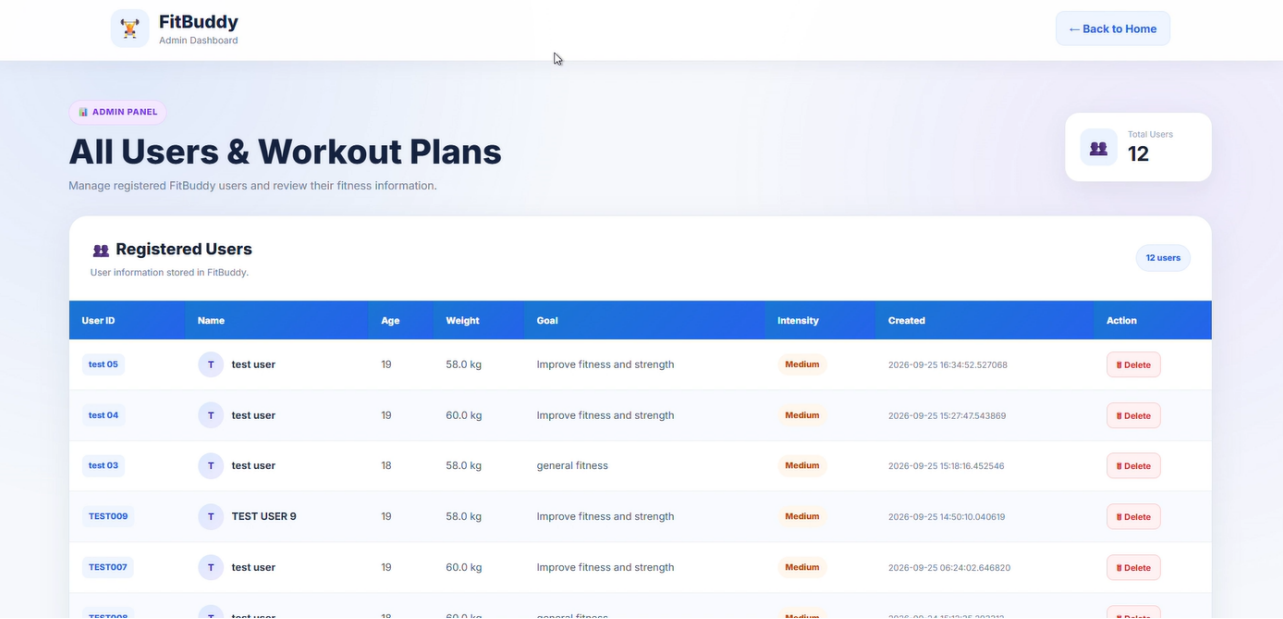


Figure 3. Current FitBuddy admin dashboard screenshot supplied for the updated project report.

The dashboard presents a total-user count and a Registered Users table. The visible columns include User ID, Name, Age, Weight, Goal, Intensity, Created time and Action. This provides a convenient overview for a project administrator or coach.

The current implementation therefore demonstrates both sides of the application: a user-facing AI planning workflow and an administrative data-management view.

12. Database, Feedback & Data Flow

Database design

The project uses SQLite for local persistence and SQLAlchemy as the ORM layer. User information and generated plan information are stored so that the admin page can display registered users and the application can retrieve an original plan when feedback is submitted.

Data	Purpose
User ID	Unique identifier used to associate profile and plan records.
Name	Displayed in the personalized result and admin view.
Age	Used as part of the profile passed to the generation logic.
Weight	Stored as part of the user's submitted profile.
Goal	Main fitness objective used in AI personalization.
Intensity	Preferred workout intensity used in plan generation.
Original plan	Preserves the first AI-generated plan.
Updated plan	Stores the revised plan after feedback.

End-to-end data flow

1. User enters profile → 2. FastAPI validates input → 3. AI modules create prompts → 4. Gemini generates content → 5. Database stores user/plan → 6. Jinja2 renders result → 7. User may submit feedback → 8. Updated plan is generated and stored.

Feedback is important because it turns the application from a one-time generator into an adaptive workflow. The original plan remains useful as a reference while the revised plan reflects the requested change.

Safety-oriented implementation

The project generation prompts are designed to provide general wellness guidance rather than medical diagnosis. For a real production deployment, health-related edge cases, professional review, authentication and stronger data protection would require additional engineering.

13. Testing, Setup & Deployment

Development environment

- Windows laptop with Python 3.12.x.
- VS Code used as the development environment.
- Project runs inside a Python virtual environment named venv.
- FastAPI application served through Uvicorn.
- SQLite used for local database persistence.
- Google GenAI SDK used for Gemini integration.

Installation and run procedure

```
# From the FitBuddy project folder
python -m venv venv
venv\Scripts\activate

python -m pip install -r requirements.txt

python -m uvicorn app.main:app --reload

Open:
http://127.0.0.1:8000

Health check:
http://127.0.0.1:8000/api/health
```

Testing performed during development

- Verified Python virtual-environment activation.
- Installed python-dotenv and Google GenAI dependencies.
- Verified that the Gemini API key was loaded from .env without exposing the key.
- Tested multiple Gemini model IDs directly before selecting a working model.
- Verified that Uvicorn starts successfully and the FastAPI application reaches startup completion.
- Verified the health endpoint returns the expected application status.
- Verified the home page, personalized result page and admin dashboard.
- Tested nutrition generation and corrected the generation call so the complete nutrition/recovery response is returned.
- Checked static CSS loading and fixed the FastAPI static-file mounting configuration.

Deployment status

The project is currently suitable for local demonstration and academic project submission. A production deployment would require additional security, authentication, secrets management, database hardening, monitoring and cloud hosting configuration.

14. Conclusion, Limitations & Future Scope

Conclusion

FitBuddy demonstrates a complete AI-enabled web application that combines a modern browser interface with FastAPI, Gemini AI and SQLite. The project accepts a user's fitness profile, generates a structured 7-day plan, supplies supporting nutrition/recovery guidance and provides a feedback mechanism for revising the plan. The admin dashboard adds a practical way to review stored user information.

The supplied reference document provided the main project concept, scenarios, architecture and development workflow. The completed implementation was then organized into a cleaner modular structure and updated with the actual screenshots supplied for this report. The model configuration was also updated during development to the working Gemini 3.5 Flash Lite model.

Limitations

- The application is primarily designed for local development and demonstration.
- SQLite is appropriate for a small local project but would need replacement or redesign for large-scale concurrent usage.
- AI output can vary and should be treated as generated guidance rather than guaranteed expert advice.
- Authentication and role-based access are limited in the current academic implementation.
- The project does not directly integrate wearable sensors or live health data.

Future Scope

- Add secure user authentication and role-based admin access.
- Deploy the application to a cloud platform with a production database.
- Add richer progress tracking and plan history.
- Integrate optional wearable or activity data through approved APIs.
- Improve structured AI output with stronger schemas and validation.
- Add automated tests and continuous integration.
- Add multilingual support for a wider range of users.
- Provide a polished PDF/export option for generated plans.

Final Project Summary

FitBuddy connects user profile → FastAPI → Gemini AI → SQLite → personalized result, with feedback and admin monitoring as supporting workflows.